



Asignatura: Lab Sistemas Operativos II

Tema del trabajo: Manual de Usuario

Integrantes:

20140083 - Eduar Moreno

122070020 - Jose Sanchez

120160026 - Erasmo Mejía

122070150 - Bryan Campigotti

122079169 - Marco Izaguirre

121390061 - Oscar Montejo

222020008 - Fernando García

322450013 - Edar Murcia

Catedrático: Ricardo E. Lagos

Manual de Usuario

Docker Security Lab

Versión: 1.0

1. Introducción

Docker Security Lab es una aplicación desarrollada en **Node.js** con **Express**, cuyo objetivo es demostrar las diferencias entre un **contenedor Docker inseguro** y uno **configurado con buenas prácticas de seguridad**.

El laboratorio incluye:

- Una aplicación web en Express.
- Un Dockerfile inseguro.
- Un Dockerfile seguro.
- Variables de entorno para demostrar el manejo de secretos.

2. Estructura del proyecto

El proyecto contiene los siguientes archivos:

```
docker-security-lab/  
├── app.js  
├── package.json  
├── Dockerfile.inseguro  
└── Dockerfile.seguro
```

Descripción

Archivo	Descripción
app.js	Aplicación principal en Express.
package.json	Dependencias del proyecto.
Dockerfile.inseguro	Imagen Docker con malas prácticas de seguridad.
Dockerfile.seguro	Imagen Docker aplicando buenas prácticas.

3. Aplicación principal (app.js)

La aplicación utiliza **Express** para crear un servidor web.

Ruta principal

GET /

- ✓ Muestra una página HTML con:
 - Un título.
 - El usuario que ejecuta el proceso.
 - Un enlace hacia la ruta **/secrets**.

Ejemplo:

Ejemplo

Usuario actual: root

Ver secretos

Ruta de secretos

GET /secrets

- ✓ Devuelve un objeto JSON con información almacenada en variables de entorno.

Ejemplo:

```
{
  "mensaje": "Estas son variables sensibles",
  "DB_PASSWORD": "...",
  "API_KEY": "...",
  "usuario_proceso": "root"
}
```

Esta ruta fue creada únicamente con fines educativos para demostrar el acceso a variables de entorno.

4. Dockerfile inseguro

El archivo **Dockerfile.inseguro** contiene una configuración con varias vulnerabilidades.

Entre ellas:

Imagen base

```
FROM node:18
```

- ✓ Utiliza una imagen completa de Node.js.

Instalación de dependencias

```
RUN npm install
```

- ✓ Instala todas las dependencias sin optimización.

Copia del proyecto

```
COPY . .
```

- ✓ Copia todos los archivos al contenedor.

Variables sensibles

- ✓ Se almacenan directamente dentro de la imagen:

```
ENV DB_PASSWORD=1234  
ENV API_KEY=...
```

Esto representa una mala práctica porque cualquier persona con acceso a la imagen puede obtener dichos valores.

Usuario

No se especifica un usuario.

El contenedor se ejecuta como:

root

Esto incrementa el riesgo en caso de compromiso del contenedor.

5. Dockerfile seguro

El archivo **Dockerfile.seguro** aplica varias recomendaciones de seguridad.

Multi-stage Build

Utiliza dos etapas.

Primera etapa:

```
FROM node:18-alpine AS builder
```

✓ Se instalan únicamente las dependencias necesarias.

Segunda etapa:

```
FROM node:18-alpine
```

✓ Se genera una imagen más pequeña.

Usuario sin privilegios

Se crea un usuario específico.

```
addgroup  
adduser
```

Posteriormente se utiliza:

```
USER nodejs
```

Con esto el contenedor deja de ejecutarse como **root**.

Copia selectiva

Solo se copian los archivos necesarios:

```
COPY --from=builder
```

Esto reduce el tamaño de la imagen.

Variables de entorno

Se configura:

```
NODE_ENV=production
```

para ejecutar la aplicación en modo producción.

6. Diferencias entre ambos Dockerfiles

Dockerfile Inseguro	Dockerfile Seguro
Ejecuta como root	Ejecuta con usuario sin privilegios
Guarda secretos dentro de la imagen	No almacena secretos sensibles
Imagen grande	Imagen Alpine más ligera
No usa Multi-stage	Usa Multi-stage Build
Instala todo	Instala solo lo necesario
Mayor superficie de ataque	Menor superficie de ataque

7. Ejecución del proyecto

✓ Construir la imagen insegura

```
docker build -f Dockerfile.inseguro -t app-insegura .
```

✓ Ejecutar

```
docker run -p 3000:3000 app-insegura
```

✓ Construir la imagen segura

```
docker build -f Dockerfile.seguro -t app-segura .
```

✓ Ejecutar

```
docker run -p 3000:3000 app-segura
```

8. Acceso a la aplicación

- ✓ Una vez iniciado el contenedor, abrir el navegador:

```
http://localhost:3000
```

- ✓ Se visualizará la página principal. Posteriormente se puede acceder a:

```
http://localhost:3000/secrets
```

- ✓ para observar las variables de entorno configuradas.

9. Buenas prácticas implementadas

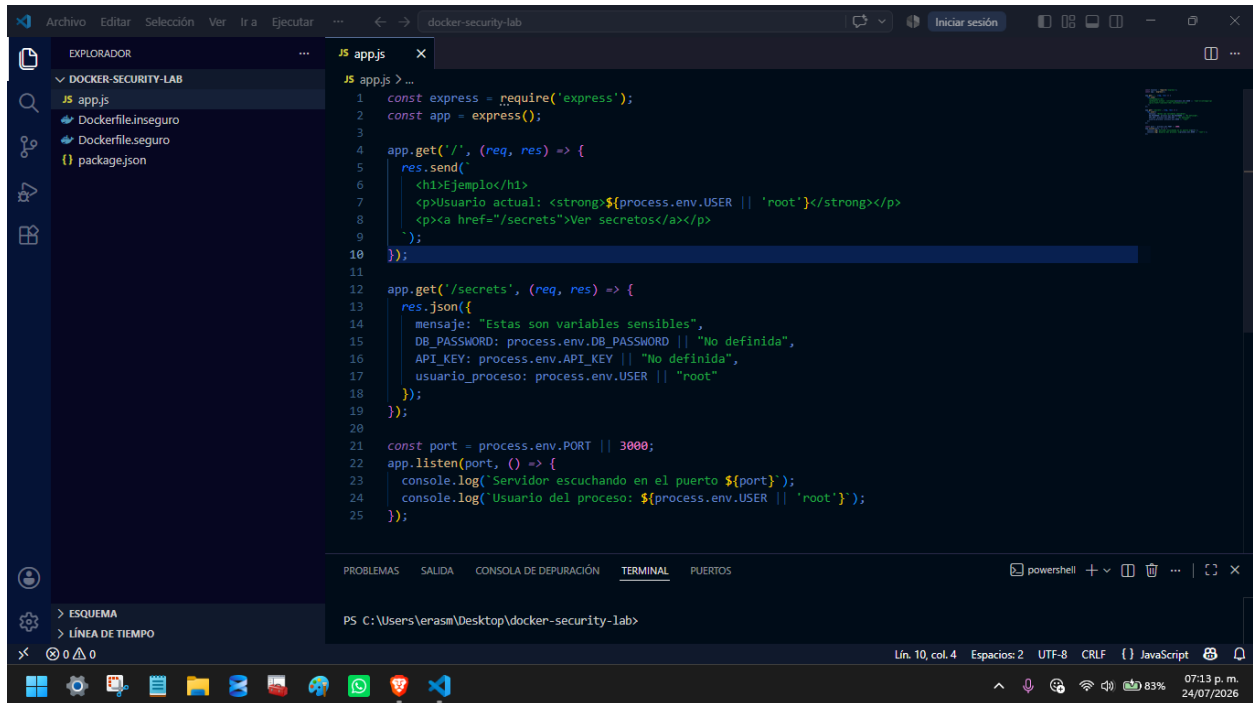
El Dockerfile seguro incorpora las siguientes recomendaciones:

- Uso de imágenes ligeras (Alpine).
- Construcción en múltiples etapas.
- Ejecución con un usuario sin privilegios.
- Instalación mínima de dependencias.
- Configuración del entorno de producción.
- Reducción del tamaño de la imagen.
- Disminución de la superficie de ataque.

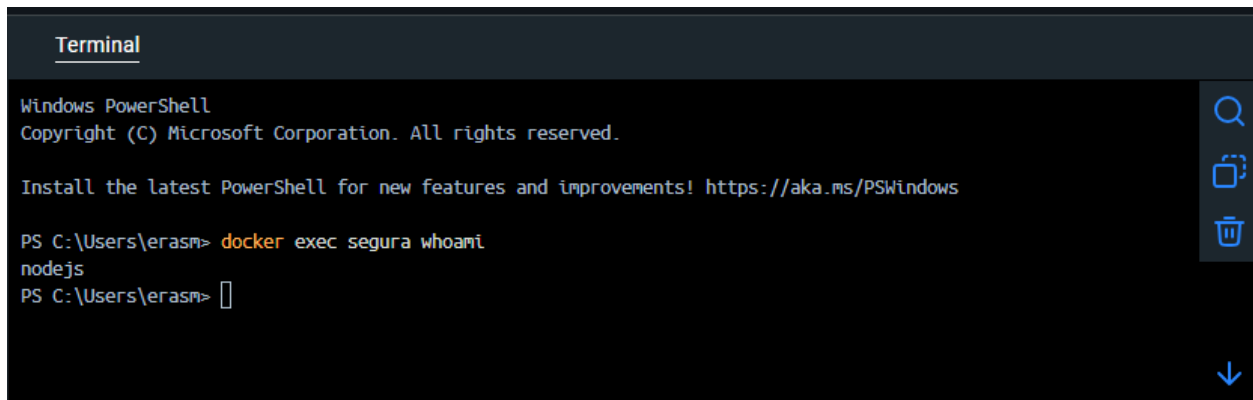
Conclusiones

Este laboratorio demuestra cómo la configuración de un contenedor Docker influye directamente en la seguridad de una aplicación. Comparando un Dockerfile inseguro con uno seguro, es posible identificar prácticas como el uso de usuarios sin privilegios, la reducción del tamaño de la imagen y la protección de la información sensible, contribuyendo a un entorno más seguro y alineado con las buenas prácticas de despliegue en contenedores.

Anexos



```
1 const express = require('express');
2 const app = express();
3
4 app.get('/', (req, res) => {
5   res.send(
6     <h1>Ejemplo</h1>
7     <p>Usuario actual: <strong>${process.env.USER || 'root'}</strong></p>
8     <p><a href="/secrets">Ver secretos</a></p>
9   );
10 });
11
12 app.get('/secrets', (req, res) => {
13   res.json({
14     mensaje: "Estas son variables sensibles",
15     DB_PASSWORD: process.env.DB_PASSWORD || "No definida",
16     API_KEY: process.env.API_KEY || "No definida",
17     usuario_proceso: process.env.USER || "root"
18   });
19 });
20
21 const port = process.env.PORT || 3000;
22 app.listen(port, () => {
23   console.log("Servidor escuchando en el puerto ${port}");
24   console.log("Usuario del proceso: ${process.env.USER || 'root'}");
25 });
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\erasm> docker exec segura whoami
nodejs
PS C:\Users\erasm>
```

Terminal

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\erasm> docker run -d --name insegura -p 3000:3000 app-insegura
07e457fa2466f4c1f4b2da24c2908fedc26284f9e4e58bbef8ee80ecf1914b92
PS C:\Users\erasm>

Terminal

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\erasm> docker exec insegura whoami
root
PS C:\Users\erasm>

A screenshot of a web browser window displaying a web application. The address bar shows 'localhost:3000'. The page has a title 'Ejemplo' and a message 'Usuario actual: root'. Below this, there is a link labeled 'Ver secretos'. The Windows taskbar is visible at the bottom of the screen, showing various application icons and the system clock indicating 07:17 p.m. on 24/07/2026.

